

Manuale d'uso — correlazioni dinamiche trimero a catena aperta versione attuale (circuitto VQE reale W-2qC.K2)

Note di lavoro — Tesi triennale, Samuele Schiavi
Relatore: Prof. Alessandro Chiesa

Scopo

Questo manuale spiega come eseguire la simulazione delle correlazioni dinamiche $C_{ij}^{\alpha\beta}(t)$ per il trimero a catena aperta nella versione *attuale*: lo stato fondamentale non è più preparato con le ampiezze esatte, ma con il circuito VQE reale (ansatz W-2qC.K2, 10 parametri, 2 cicli), ottimizzato numericamente. La versione a stato esatto resta disponibile (comportamento di default se non si passano parametri VQE) e serve da riferimento: vedi il manuale gemello `manuale_uso_correlazioni_trimero_catena_statevector.tex`.

Prerequisiti

Tutti i file del manuale gemello, più:

- `vqe_w2qC_k2_trimero_catena.py` — costruzione dell'ansatz W-2qC.K2 e ottimizzazione VQE, su **entrambi** i punti di lavoro.
- `validate_vqe_circuito_correlazioni_catena.py` — confronto sistematico contro la versione a stato esatto, entrambi i punti.

Importante, a differenza dell'anello: esistono **due** file di parametri ottimali, uno per punto di lavoro — `w2qC_k2_params_vqedm.npz` (VQE-DM, $J=1, b=3.0, D=0.15$) e `w2qC_k2_params_S0.npz` (S0, $J=1, b=3.0073414, D=0.3$). Come nell'anello, i parametri dipendono dal punto: non riusare quelli di un punto per l'altro.

Passo 1 — ottimizzazione VQE (una tantum per punto di lavoro)

```
python vqe_w2qC_k2_trimero_catena.py
```

Esegue 60 partenze casuali (COBYLA) più polish (L-BFGS-B) su **entrambi i punti in sequenza** (funzione `main()`), e salva due file. Valori ottenuti:

	VQE-DM	S0
E_{vqe}	-7.36924699164432	-7.75658090425334
E_{esatto}	-7.36924699164433	-7.75658090425337
$\mathcal{F}$	1.000000000000	0.99999999999992

Nella sessione di sviluppo sono bastati 15 restart per raggiungere questi valori (verificato: 5 restart già sufficienti al punto VQE-DM); lo script di produzione ne usa 60 per sicurezza, seguendo la stessa cautela già documentata in `trimero_catena_vqe_dm.tex` per il punto $b = b_c$ (vicino a un level crossing a $D = 0$, dove pochi restart possono dare un falso ceiling). Se il tuo output mostra $1 - \mathcal{F}$ molto più grande (es. 10^{-3} o peggio), aumenta `n_restarts` nella chiamata a `optimize_point`.

Passo 2 — un correlatore singolo, con preparazione VQE

```
import numpy as np
from circuito_correlazioni_trimero_catena import correlator_from_circuit

data = np.load("w2qC_k2_params_vqedm.npz")
params = data["params"]

J, b, D = 1.0, 3.0, 0.15
C = correlator_from_circuit(1, 'y', 1, 'y', t=1.3, N=200, J=J, b=b, D=D,
                           ansatz_params=params)

print(C)
```

Passando `ansatz_params`, la preparazione usa `w2qC_k2_circuit()` (X + due cicli di [2 blocchi W, 3 R_y]) invece di `prepare_state`. Per il punto S0, caricare `w2qC_k2_params_S0.npz` e usare $b=3.0073414$, $D=0.3$.

Passo 3 — validazione contro la versione a stato esatto

```
python validate_vqe_circuito_correlazioni_catena.py
```

Richiede **entrambi** i file `.npz` del Passo 1 (già presenti). Esegue automaticamente entrambi i punti in sequenza, ricalcola le 81 combinazioni con preparazione VQE e con preparazione esatta, stampa l'errore medio e massimo:

	VQE-DM	S0
errore medio	1.9×10^{-8}	3.8×10^{-8}
errore massimo	5.9×10^{-8}	9.8×10^{-8}
$\sqrt{1 - \mathcal{F}}$ (riferimento)	4.5×10^{-8}	9.1×10^{-8}

Entrambi coerenti con $\sqrt{1 - \mathcal{F}}$, come atteso per un residuo di fedeltà a livello di stato. Errori molto più grandi indicano parametri VQE non ottimizzati bene (torna al Passo 1).

Passo 4 — scan completo (statevector + shot), preparazione VQE

A differenza dell'anello, non esiste uno script `scan81_vqe_*` separato: lo scan completo con preparazione VQE è incorporato nei notebook (vedi sotto), non in uno script standalone. Per un confronto rapido da riga di comando, usare `correlator_from_circuit` con `ansatz_params` in un ciclo sulle 81 combinazioni (Passo 2, ripetuto).

Come esplorare tutto insieme: il notebook

`circuito_correlazioni_trimero_catena_vqe.ipynb` carica i parametri già ottimizzati (Passo 1, non li ricalcola) e mostra, con la preparazione VQE al posto di `prepare_state`: misura diretta delle 81 combinazioni, heatmap $|C|$ d'insieme, vista nel tempo su griglia 9×9 (Re e Im), selettore per ispezione singola, verifica della relazione P_{13} ($C_{11} = C_{33}$), spettro dietro un caso ricco/piatto, confronto con lo scan a shot finiti precomputato (`scan81_catena_vqedm_results.npz`, Passo 4 del manuale gemello). Lavora solo al punto VQE-DM (hard-coded); per S0 vanno adattati i parametri (J, b, D) e ricaricato `w2qC_k2_params_S0.npz`.

Cosa NON cambia rispetto alla versione a stato esatto

`build_correlator_circuit` e `correlator_from_circuit` sono retrocompatibili: se non si passa `ansatz_params` (o si passa `None`), il comportamento è identico, bit per bit, alla versione `stat-evector` — tutto il codice esistente (`scan81_trimero_catena.py`, `validate_shot_noise_trimero_catena.py`, i primi tre notebook della fase) continua a funzionare senza modifiche.

Problemi comuni

- `FileNotFoundError` su uno dei due file `w2qC_k2_params_*.npz` — va generato eseguendo `vqe_w2qC_k2_trimero_catena.py` (Passo 1); richiede alcuni minuti per via dei 60 restart $\times 2$ punti.
- Fidelity bassa dopo l'ottimizzazione — vedi nota al Passo 1 (minimo locale, più probabile vicino a b_c); aumentare i restart di solito risolve.
- Uso dei parametri sbagliati per il punto — verificare sempre quale `.npz` si sta caricando: i due punti hanno parametri ottimali diversi e non intercambiabili.
- Import circolare se si prova a importare `ground_state` da `circuito_correlazioni_trimero_catena.py` dentro `vqe_w2qC_k2_trimero_catena.py` — per questo il secondo modulo definisce una propria copia locale `_ground_state`, stessa scelta già fatta per l'anello.

Riferimenti

Derivazione dell'ansatz: `trimero_catena_vqe_dm.tex`, `ansatz_catena.py`. Documentazione teorica: `simmetrie_correlatori_trimero_catena.tex`, `circuito_correlazioni_trimero_catena_spiegato.tex`. Catalogo completo della fase: `trimero_catena_correlazioni.tex`.